Politechnika Świętokrzyska

Aplikacje Internetowe
Podstawy Technologii Internetowych

Dr inż. **Wikarek Jarosław**
Katedra Informatyki Stosowanej
e-mail: j.wikarek@tu.kielce.pl
pokój: d408

Wydział Elektrotechniki, Automatyki i Informatyki

PHP

Interpretowany, skryptowy język programowania zaprojektowany do generowania stron internetowych i budowania aplikacji webowych w czasie rzeczywistym.

Podstawa języka

Zasada działania

- Parser PHP przetwarzając stronę ma 2 tryby pracy:
  - ✓ HTML – Gdzie cała treść jest wyświetlana, bez przetwarzania.
  - ✓ PHP – Gdzie treść jest traktowana jako skrypt do przetworzenia.
- Do określenia w pliku, co jest kodem HTML a co PHP służą specjalne znaczniki. Początkowo parser jest w trybie HTML. Aby przejść do trybu PHP można użyć jednego ze znaczników:
  - ✓ `<? Kod w PHP; ?>`
  - ✓ `<?php Kod w PHP; ?>`
  - ✓
```
<script language="php">
   Kod w PHP;
</script>
```

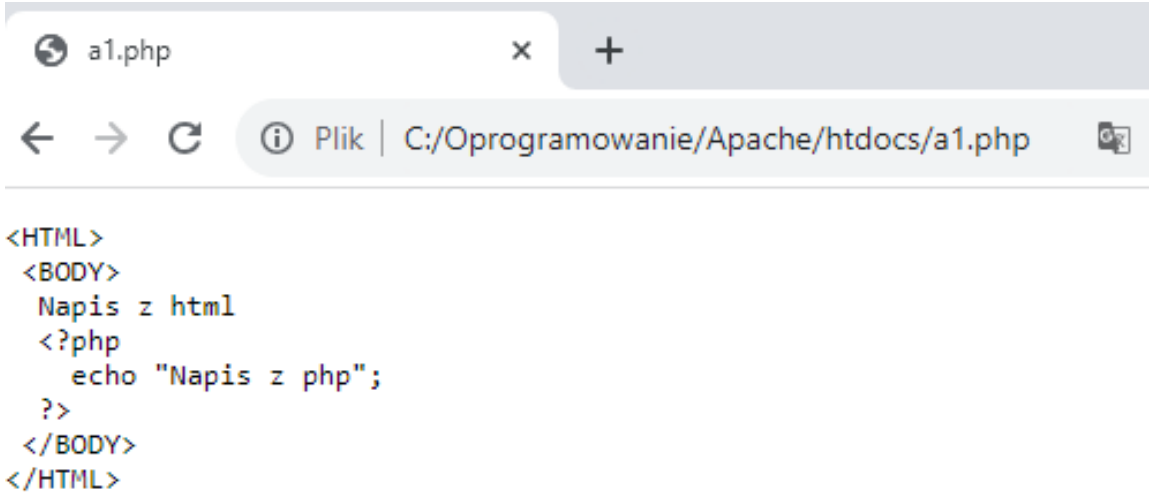
Zasada działania

- Przykład 1

```
<HTML>
 <BODY>
  Napis z html
  <?php
    echo 'Napis z php';
  ?>
 </BODY>
</HTML>
```

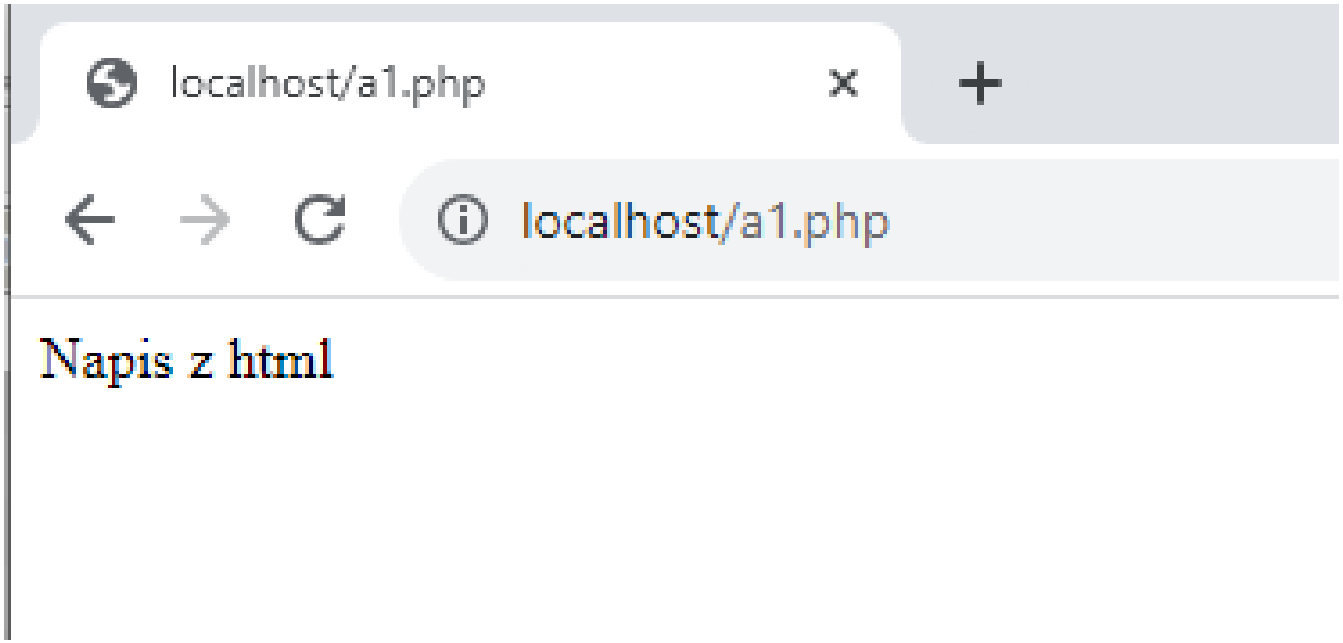
Zasada działania

- Przykład 1 – uruchamianie
  - ✓ Błędne jako plik lokalny



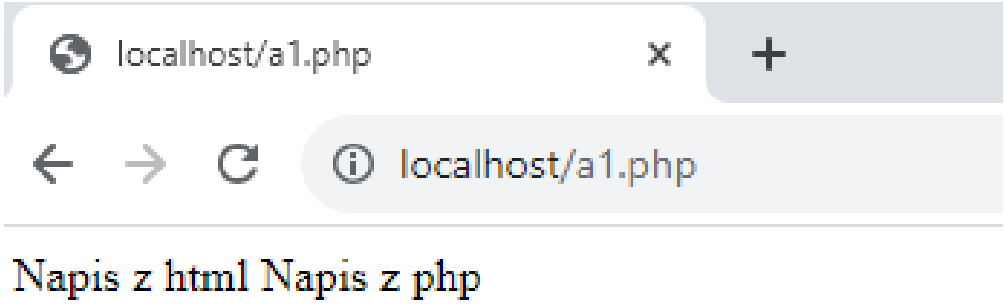
Zasada działania

- Przykład 1 – uruchamianie
  - ✓ Błędne - Nie przez serwer lub serwer nie obsługuje PHP



Zasada działania

- Przykład 1 – uruchamianie
  - ✓ Serwer obsługuje PHP

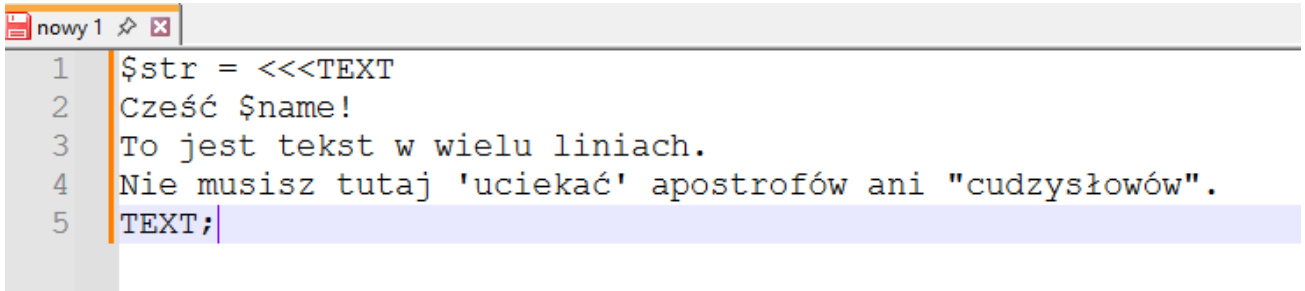


Składnia języka

- Oddzielanie instrukcji – Instrukcje są oddzielane tak samo jak w C czy Perlu - należy kończyć każde wyrażenie średnikiem. Znacznik zamykający (?>) także kończy instrukcję.
- Komentarze . W PHP występują trzy rodzaje komentarzy:
  - ✓ `/* Komentarze w stylu języka C */`
  - ✓ `// Komentarze w style języka C++`
  - ✓ `# Komentarze w stylu skryptów powłoki`

Typy danych

- Cztery skalarne:
  - ✓ boolowski (boolean).
  - ✓ liczba całkowita (integer)
  - ✓ liczba zmiennoprzecinkowa (float) (1.234; 1.2e3; 7E-10;)
  - ✓ Łańcuch znaków (string):
    - apostrofy.
    - Cudzysłowy.
    - Heredoc (<<<identyfikator),



Typy danych

- Dwa typy złożone:
  - ✓ tablica (array).
  - ✓ obiekt (object).
- Dwa typy specjalne:
  - ✓ identyfikator zasobów (resource).
  - ✓ NULL (null).

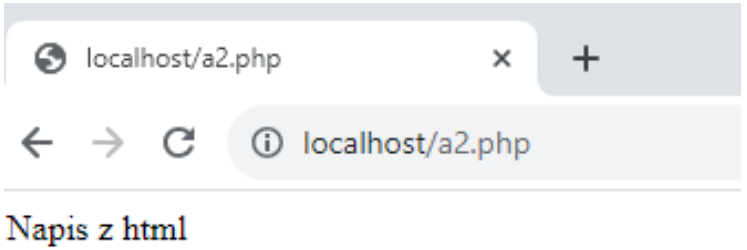
Zmienne

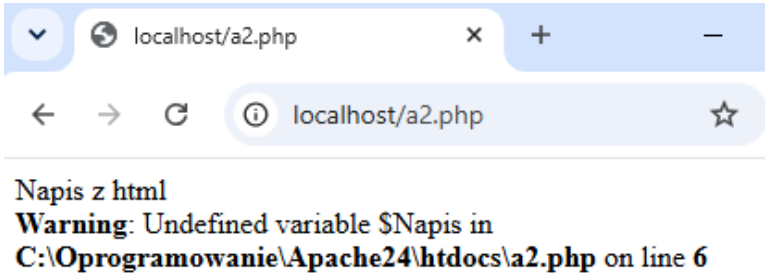
- PHP nie wymaga otwartego deklarowania zmiennych.
- PHP automatycznie deklaruje zmienną w momencie, w którym po raz pierwszy zostanie przypisana jej jakaś wartość.
- Aby sprawdzić typ i wartość konkretnego wyrażenia należy użyć funkcji **`var_dump()`**.
- Narzucenie konwersji zmiennej do określonego typu wymaga rzutowania zmiennej lub użycia funkcji **`settype()`**.
- W języku PHP wszystkie nazwy zmiennych zaczynają się od znaku dolara ($).
- W PHP wielkość liter w nazwach zmiennych ma istotne znaczenie.

Zmienne

- Przykład 2.

```
<HTML>
 <BODY>
   Napis z html
   <?php
     $napis = "Napis w PHP";
     echo $Napis ;
   ?>
  </BODY>
</HTML>
```

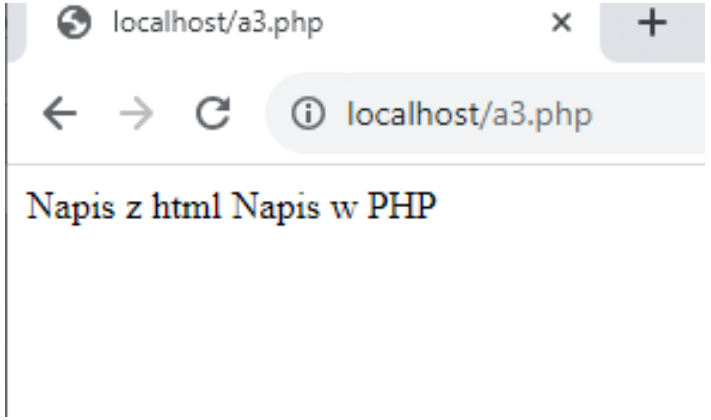




Zmienne

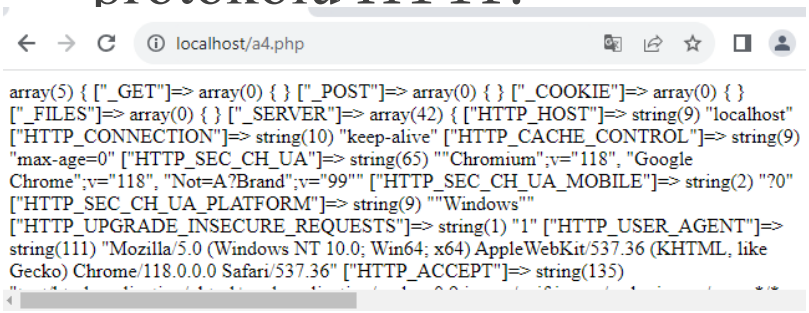
- Przykład 2.

```
<HTML>
 <BODY>
   Napis z html
   <?php
     $napis = "Napis w PHP";
     echo $napis ;
   ?>
  </BODY>
</HTML>
```



Zmienne predefiniowane

- **`$GLOBALS`** – Zawiera referencje do każdej ze zmiennych aktualnie dostępnych w zasięgu globalnym skryptu. Kluczami tablicy są nazwy zmiennych globalnych.
- **`$_SERVER`** – Zmienne tworzone przez serwer lub bezpośrednio powiązane ze środowiskiem uruchomieniowym danego skryptu.
- **`$_GET`** – Zmienne dostarczone do skryptu za pomocą metody GET protokołu HTTP.
- **`$_POST`** – Zmienne dostarczone do skryptu metodą POST protokołu HTTP.

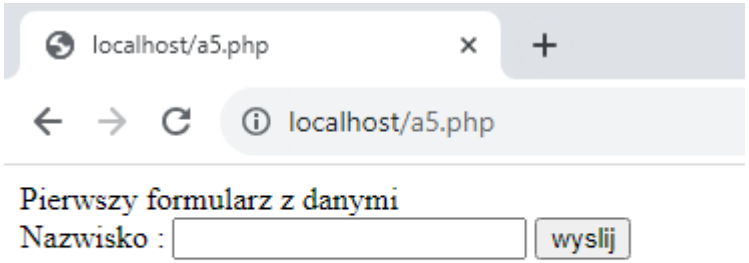


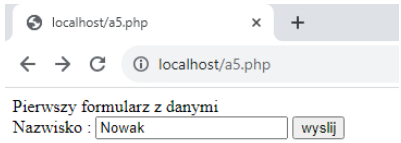


Zmienne predefiniowane

- Przykład 3.

```
<HTML>
 <BODY>
  Pierwszy formularz z danymi
  <form action="a6.php" method="GET">
    Nazwisko : <input type='text'
      name='pole_1'/>
   <input type='submit' value='wyslij'>
  </form>
 </BODY>
</HTML>
```



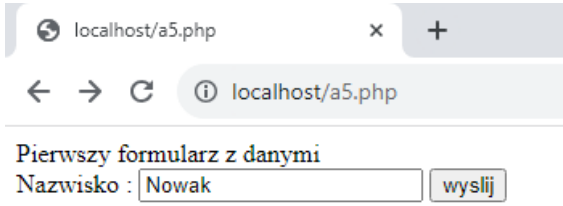


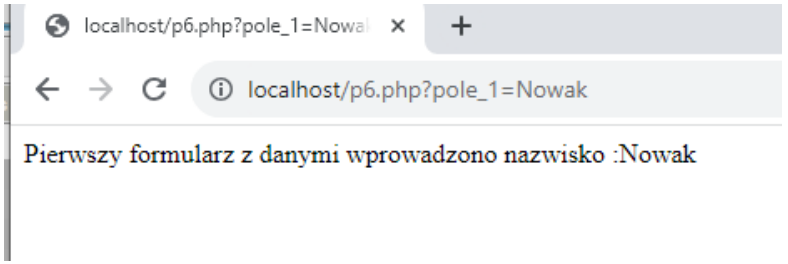
Zmienne predefiniowane

- Przykład 3.

```
<HTML>
 <BODY>
   Pierwszy formularz z danymi
   <?php
    echo "wprowadzono nazwisko :";
    echo $_GET['pole_1'];
   ?>
 </BODY>
</HTML>
```

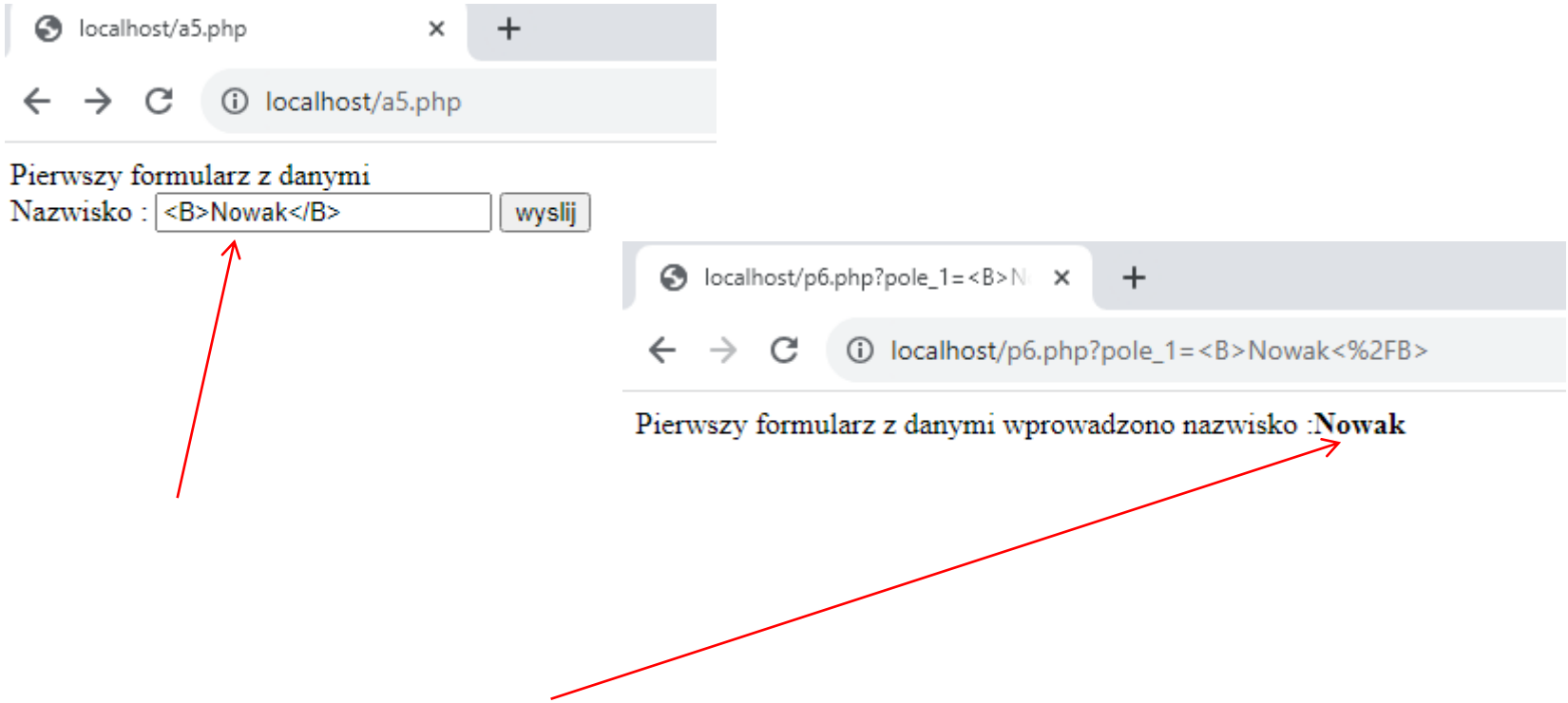
```
<HTML>
 <BODY>
   Pierwszy formularz z danymi
   <form action=”a6.php" method="GET">
    Nazwisko : <input type='text' name='pole_1'/>
     <input type='submit' value='wyslij'>
   </form>
 </BODY>
</HTML>
```





Zmienne predefiniowane

- **ATAK XSS**



Zmienne predefiniowane

- **`$_COOKIE`** – Zmienne dostarczone do skryptu przez ciasteczka HTTP.
- **`$_FILES`** – Zmienne dostarczone do skryptu przez przesłanie plików do serwera metodą POST protokołu HTTP.
- **`$_ENV`** – Zmienne dostarczone do skryptu przez środowisko operacyjne.
- **`$_REQUEST`** – Zmienne użytkownika dostarczone do skryptu przez każdy mechanizm wejścia.
- **`$_SESSION`** – Zmienne aktualnie zarejestrowane jako sesyjne.
- **`$GATEWAY_INTERFACE`** – Informacja o specyfikacji CGI używanej przez serwer, np. 'CGI/1.1'.

Zmienne predefiniowane – środowisko serwera

- **`$SERVER_NAME`** – Nazwa hosta serwera na którym skrypt jest uruchamiany. Jeśli skrypt pracuje na wirtualnym hoście, to zmienna przyjmie jako wartość nazwę wirtualnego hosta.
- **`$SERVER_SOFTWARE`** – Ciąg identyfikujący serwera podawany przy odpowiadaniu na zapytania.
- **`$SERVER_PROTOCOL`** – Nazwa i numer wersji protokołu za pomocą którego wysłano zapytanie o stronę, np. 'HTTP/1.0'.
- **`$REQUEST_METHOD`** – Metoda zapytania użyta do uzyskania dostępu do strony, np. 'GET', 'HEAD', 'POST', 'PUT'.
- **`$QUERY_STRING`** – Ciąg zapytania (jeśli takowy istnieje) za pomocą którego połączono się ze stroną.

Zmienne predefiniowane – środowisko serwera

- **$DOCUMENT_ROOT** – Katalog główny drzewa dokumentów, spod którego skrypt jest wykonywany - jest to ustawienie z pliku konfiguracyjnego serwera.
- Protokołu HTTP
  - ✓ $HTTP_ACCEPT
  - ✓ $HTTP_ACCEPT_CHARSET
  - ✓ $HTTP_ENCODING
  - ✓ $HTTP_ACCEPT_LANGUAGE
  - ✓ $HTTP_CONNECTION
  - ✓ $HTTP_HOST
  - ✓ $HTTP_REFERER

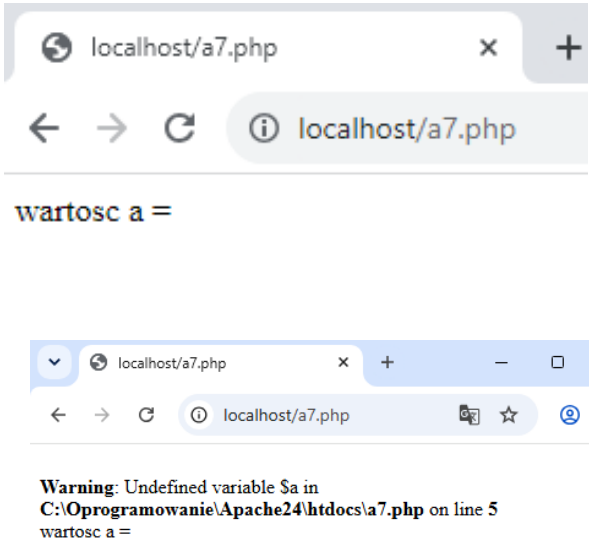
Zmienne predefiniowane

- **`$REMOTE_ADDR`** – Adres IP, z którego użytkownik ogląda bieżącą stronę.
- **`$REMOTE_PORT`** – Port używany do komunikacji pomiędzy użytkownikiem a serwerem.
- **`$SERVER_PORT`** – Port na serwerze, którego użyto do połączenia. Dla normalnych połączeń będzie to '80'.

Zasięg zmiennych

- Zasięg zmiennej jest zależny od miejsca, w jakim została zdefiniowana. Najczęściej zmienne PHP widoczne są tylko w pojedynczym zasięgu. Taki zasięg rozciąga się na pliki dołączane funkcjami `include()` i `require()`.

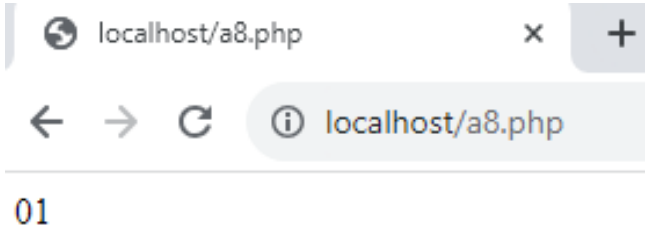
```
$a = 1
function Test()
{
 echo "wartosc a = $a";
}
Test();
```



Zasięg zmiennych

- Zasięg zmiennej jest zależny od miejsca, w jakim została zdefiniowana. Najczęściej zmienne PHP widoczne są tylko w pojedynczym zasięgu. Taki zasięg rozciąga się na pliki dołączane funkcjami `include()` i `require()`.

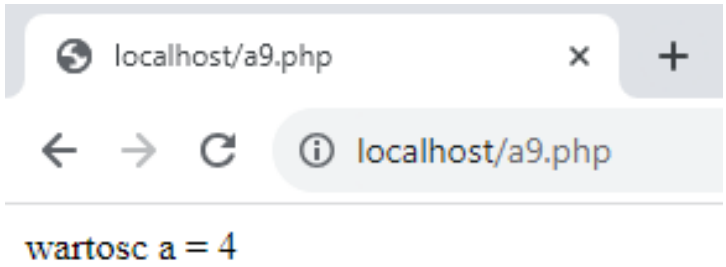
```
function Test()
{
 static $a = 0;
 echo $a;
 $a++;
}
Test();
Test();
```



Zasięg zmiennych

- Zasięg zmiennej jest zależny od miejsca, w jakim została zdefiniowana. Najczęściej zmienne PHP widoczne są tylko w pojedynczym zasięgu. Taki zasięg rozciąga się na pliki dołączane funkcjami `include()` i `require()`.

```
$a = 4;
function Test()
{
 global $a;
 echo "wartosc a = $a";
}
Test();
```



Zasięg zmiennych

- Kiedy do skryptu PHP zostanie wysłany formularz, każda zmienna z tego formularza zostanie automatycznie dostarczona do tego skryptu przez PHP. Jeśli dyrektywa track_vars jest włączona, to zmienne te będą umieszczone w tablicach asocjacyjnych:
  - `$_POST` zmienne wysłane metodą POST.
  - `$_GET` zmienne wysłane metodą GET.
  - `$_FILES` plik wysłane metodą POST.

Zmienne z poza PHP

- Przy tworzeniu formularza, można użyć obrazka, zamiast standardowego przycisku Wyślij, przy pomocy następującego znacznika:

```
<input type="image" src="image.gif" name="sub">
```

Kiedy użytkownik kliknie gdzieś na obrazku, formularz, którego to dotyczy, zostanie wysłany do serwera z dwiema dodatkowymi zmiennymi: `sub_x` i `sub_y`.

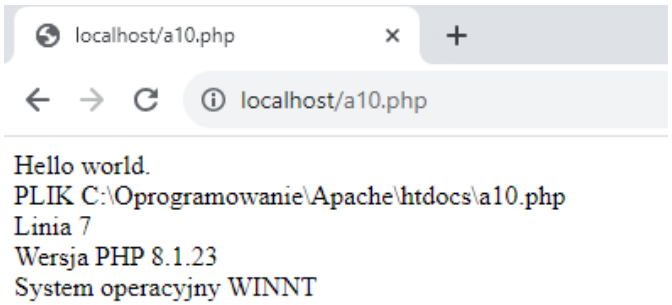
Stałe

- Stałe predefiniowane
  - **__FILE__** – Nazwa pliku ze skryptem, który jest aktualnie przetwarzany.
  - **__LINE__** – Numer linii w skrypcie, która aktualnie jest przetwarzana.
  - **PHP_VERSION** – Ciąg reprezentujący wersję parsera PHP aktualnie używaną.
  - **PHP_OS** – Nazwa systemu operacyjnego.
  - **TRUE** – Logiczna wartość prawdy.
  - **FALSE** – Logiczna wartość fałszu.

Stałe

- Definicja stałej

```
<?php
 define("STALA", "Hello world. ");
 echo STALA;
 echo "<BR> ";
 echo "PLIK  ".__FILE__ ;
 echo "<BR> ";
 echo "Linia  ".__LINE__ ;
 echo "<BR> ";
 echo "Wersja PHP  ".PHP_VERSION ;
 echo "<BR> ";
 echo "System operacyjny  ".PHP_OS ;
?>
```



Operatory

- Arytmetyczne
  - ✓ `$a+$b` Dodawanie Suma Sa i Sb.
  - ✓ `$a-$b` Odejmowanie Różnica Sa i Sb.
  - ✓ `$a*$b` Mnożenie Iloczyn Sa i Sb.
  - ✓ `$a/$b` Dzielenie Iloraz Sa i Sb (bez reszty).
  - ✓ `$a%$b` Modulo Reszta z dzielenia Sa przez Sb.

Operatory

- Przypisania
  - ✓ `$a+=2` Do zmiennej $a dodane zostanie 2
  - ✓ `$a-=2` Od zmiennej $a odjęte zostanie 2
  - ✓ `$a*=2` Zmienna $a zostanie pomnożona przez 2
  - ✓ `$a/=2` Zmienna $a dodane podzielona przez 2
  - `$a%=2` Zmienna $a przyjmie wartość reszty z dzielenia $a przez 2
  - ✓ `$a.=" dalszy ciąg"` Do zmiennej $a na końcu dodany zostanie ciąg "dalszy ciąg"

Operatory

- Porównania
  - ✓ `$a==$b` Prawda jeśli $a jest równe $b.
  - ✓ `$a!=$b` Prawda jeśli $a nie jest równe $b.
  - ✓ `$a<$b` Prawda jeśli $a jest mniejsze niż $b.
  - ✓ `$a>$b` Prawda jeśli $a jest większe niż $b.
  - ✓ `$a<=$b` Prawda jeśli $a jest mniejsze lub równe $b.
  - ✓ `$a>=$b` Prawda jeśli $a jest większe lub równe $b.

Operatory

- Inkrementacji i dekrementacji
  - `++$a` Preinkremetacja a Zwiększa $a o jeden, a następnie zwraca $a.
  - `$a++` Postinkrementacja a Zwraca $a a następnie zwiększa $ao jeden.
  - `–$a` Predekrementacja a Zmniejsza $a o jeden, po czym zwraca $a.
  - `$a–` Posdekrementacja a Zwraca $a. po czym zmniejsza $a o jeden.
- Logiczne
  - `$a&&$b` AND Prawda, jeśli $a i $b są prawdą
  - `$a||$b` OR Prawda, jeśli $a lub 4b są prawdą
  - `! $a` NOT Prawda, jeśli $a nie jest prawdą

Operatory

- Inne operatory
  - ✓ Operator wywołania

    ```
    `ls -al `
    ```
  - ✓ Operator ciągu

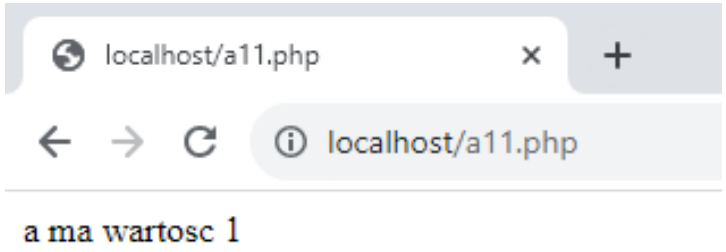
    .
  - ✓ Operator kontroli błędów

    @

Instrukcje

- Instrukcja warunkowa

```
<?
 if(wyrażenie_warunkowe)
  instrukcja
else
  instrukcja
?>
```

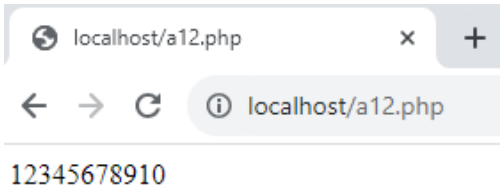



```
<HTML>
 <BODY>
   <?php
    $a=1;
    if ($a==1)
     echo ”a ma wartosc 1”;
    else
     echo ”a ma inna wartosc”;
   ?>
 </BODY>
</HTML>
```

Instrukcje

- Pętla FOR

```
<?
 for(inicjalizacja; warunek; modyfikacja)
 {
    blok wyrażeń
 }
?>
```




```
<HTML>
 <BODY>
   <?php
    for ($a=1;$a<=10;$a++)
     echo $a ;
   ?>
 </BODY>
</HTML>
```

Instrukcje

- Pętla WHILE

```
<?
 while (warunek)
    {
      instrukcje
    }
?>
```

Instrukcje

- Pętla DO WHILE

```
<?
     do
       {
         instrukcje
       }
       while( warunek );
?>
```

- Continue
- Break

Instrukcje

- SWITCH

```
switch($zmienna)
{
   case 'wartość1':
      instrukcje; break;
   case 'wartość2':
      instrukcje; break;
   default:
      instrukcje;
}
```

Tablice

- Przypisanie wartości

```
$tablica[0] = ”Wpis numer 0 ”
$tablice[] = ”Wpis numer 1 ”
```

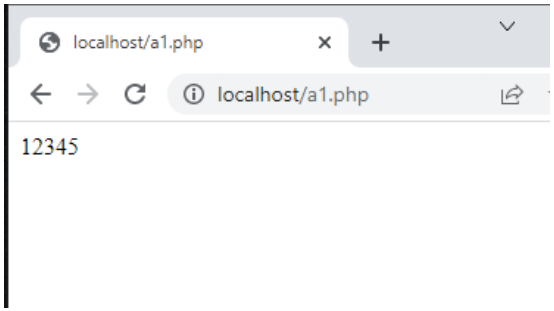
- Tablice asocjacyjne

```
$tablica[”imie”] = ”Jan” ;
$tablica[”nazwisko”] = ”Kowalski ” ;
$tablica[”adres”] = ”Polna 1 ”;
$a=array(”a”=>”jabłko”,”b”=>”banan”);
```

Tablice

- Przeglądanie zawartości
  - ✓ Znana liczba elementów

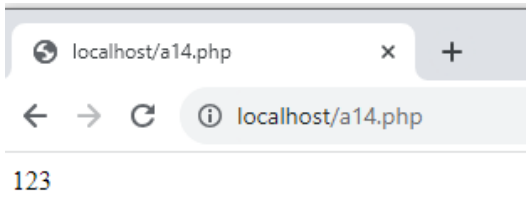
```
$tbl[]=1; $tbl[]=2; $tbl[]=3;
  $tbl[]=4;$tbl[]=5;
for( $x = 0; $x < 5; $x++ ) {
 echo $tbl[$x];
}
```



  - ✓ Nieznana liczba elementów

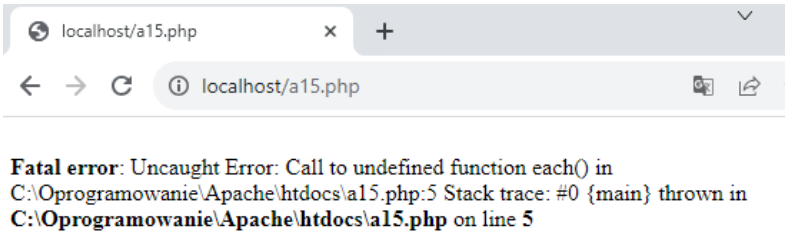
```
$tbl[]=1; $tbl[]=2; $tbl[]=3;

 for( $x= 0; $x < count($tbl); $x++ )
 { echo $tbl[$x];}
```



Tablice

- Przeglądanie zawartości
  - ✓ Tablice asocjacyjne (PHP 5)




```
<?php
 $tablica[”imie”] = ”Jan”;
 $tablica[”nazwisko”] = ”Kowalski”;
 $tablica[”adres”] = ”Polna 1”;
  while(list($klucz,$wartosc)=each($tablica))
  echo ”$klucz => $wartosc <BR>”;
?>
```

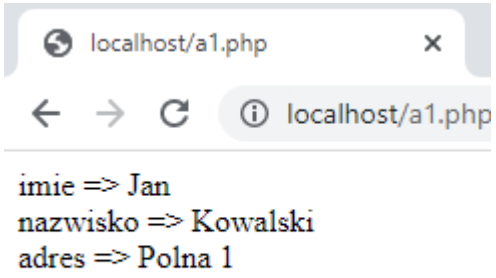
Tylko PHP 7

Funkcja **reset** ustawia wskaźnik ponownie na pierwszym elemencie tablicy

Tablice

- Przeglądanie zawartości
  - ✓ Tablice asocjacyjne (PHP 7)

```
<?php
 $tablica[”imie”] = ”Jan”;
 $tablica[”nazwisko”] = ”Kowalski”;
 $tablica[”adres”] = ”Polna 1”;
 foreach($tablica as $klucz=>$wartosc )
  {echo ”$klucz => $wartosc <BR>”;}
?>
```

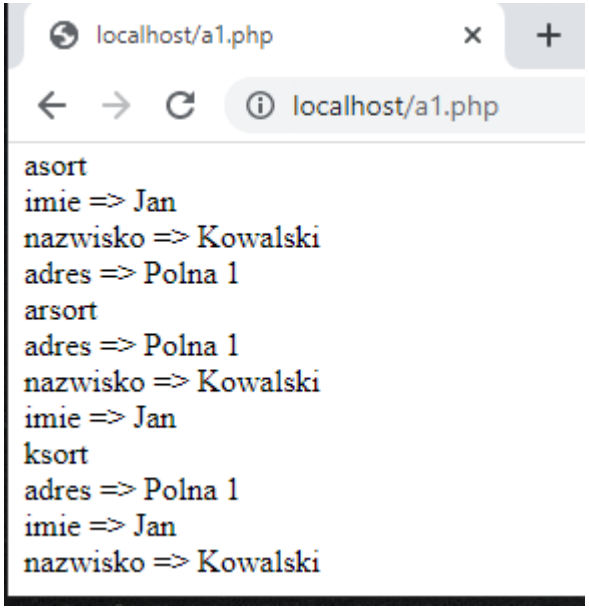


Obsługa tablic

- **asort()** – Sortuje tablice asocjacyjne zachowując przypisanie kluczy do wartość.
- **arsort()** – Sortuje w odwrotnej kolejności tablice asocjacyjne zachowując przypisanie kluczy do wartości. Funkcja prawie identyczna jak poprzednia, tyle, że dane sortowane są "od tyłu”.
- **ksort()** – Sortuje tablice asocjacyjne według kluczy.
- **rsort()** – Sortuje zwykłe tablice (nie asocjacyjne) w odwróconej kolejności.
- **sort()** – Sortuje zwykłe tablice (nie asocjacyjne) w kolejności alfabetycznej.

Obsługa tablic

```php
<?php
  $tablica["imie"] = "Jan";
  $tablica["nazwisko"] = "Kowalski";
  $tablica["adres"] = "Polna 1";
  foreach($tablica as $klucz=>$wartosc )
  {echo "$klucz => $wartosc <BR>";}
  echo "asort <BR>";
  asort($tablica);
  foreach($tablica as $klucz=>$wartosc )
  {echo "$klucz => $wartosc <BR>";}
  echo "arsort <BR>";
  arsort($tablica);
  foreach($tablica as $klucz=>$wartosc )
  {echo "$klucz => $wartosc <BR>";}
  echo "ksort <BR>";
  ksort($tablica);
  foreach($tablica as $klucz=>$wartosc )
  {echo "$klucz => $wartosc <BR>";}
?>
```



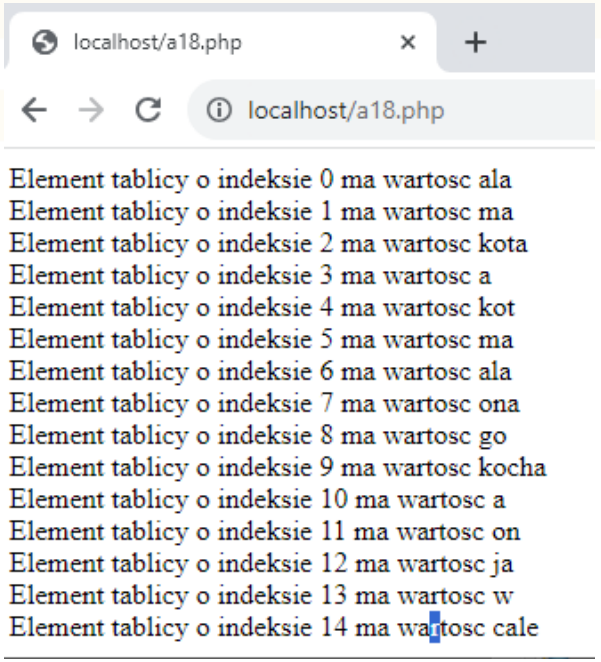
Obsługa tablic

- **`explode()`** – Rozdziela ciągów na tablicę. Jako pierwszy parametr trzeba do niej podać znak lub dłuższy ciąg, który oddziela kolejne pola, jako drugi ciąg do rozdzielenia.
- **`implode()`** – Złącznie pól tablicy w jeden ciąg, w którym pola oddzielone są jakimś znakiem (lub kilkoma). Jako pierwszy parametr podawany jest ciąg za pomocą którego "sklejane" są elementy tablicy, a jako drugi właśnie tablica do posklejania.

Obsługa tablic

- Przykład

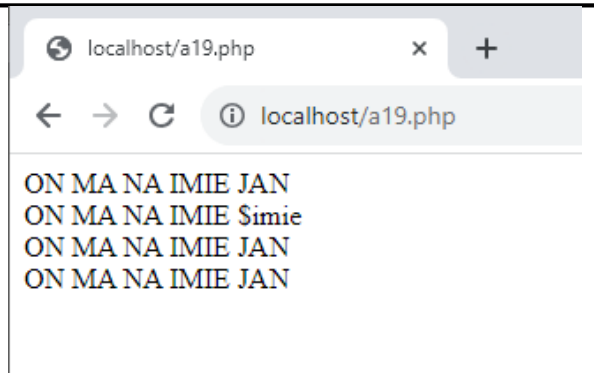
```
<HTML>
 <BODY>
  <?php
   $napis="ala ma kota a kot ma ala ona go kocha a on ja w cale";
   $tablica=explode(" ",$napis);
   for($i=0; $i<count($tablica); $i++)
    echo "Element tablicy o indeksie $i ma
     wartosc $tablica[$i]<BR>";
  ?>
 </BODY>
</HTML>
```



Programowanie

- Zmienne w stringach

```
<?php
    $imie='JAN';
    $napi_1=”ON MA NA IMIE $imie”;
    $napi_2=’ON MA NA IMIE $imie’;
    $napi_3=”ON MA NA IMIE ”.$imie;
    $napi_4=’ON MA NA IMIE ’.$imie;
    echo $napi_1;     echo "<BR>";
    echo $napi_2;     echo "<BR>";
    echo $napi_3;     echo "<BR>";
    echo $napi_4;
  ?>
```



Programowanie

- Funkcje
  - Funkcja jest kawałkiem kodu, który można wywołać podając jej nazwę.
  - Już wcześniej podane zostało kilka funkcji dostarczanych przez PHP, takich jak `array()`, `each()`, `list()`, `explode()` czy `implode()`. Jak można zauważyć, funkcje rozpoznaje się po nawiasach występujących po ich nazwie.
  - W nawiasach tych (jeśli funkcja tego wymaga) podaje się parametry dla danej funkcji.

Programowanie

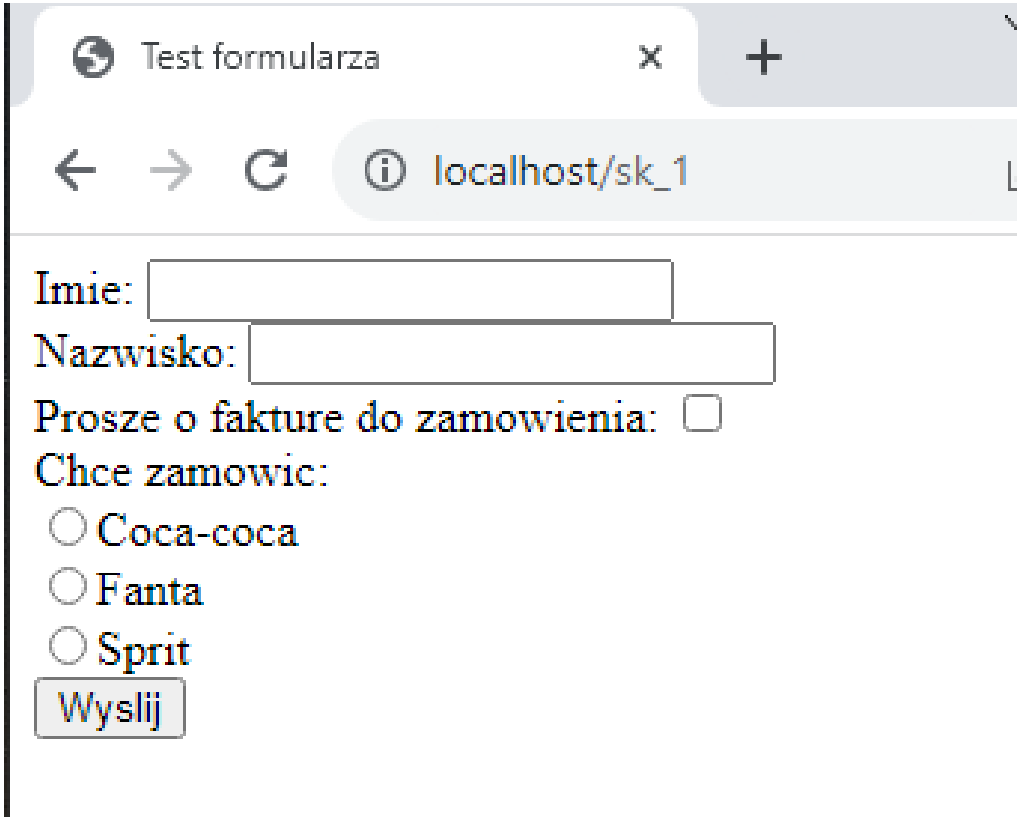
- Przykład

```
<?php
 function suma($parametr1=0,$parametr2=0)
  {
   $wartosc = $parametr1 + $parametr2;
   return $wartosc;
  }
?>
```

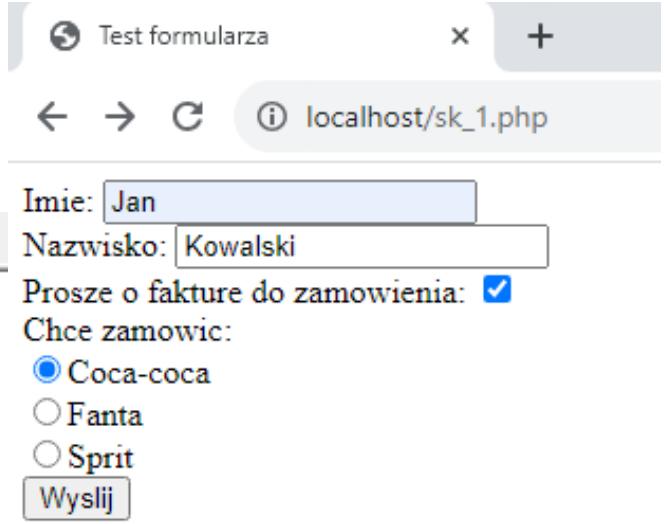
Programowanie

- Włączenie zbiorów
  - ✓ **`include plik`** – Pliki dołączane są w trakcie parsowania.
  - ✓ **`require plik`** – Pliki dołączane są do kodu jeszcze przed parsowaniem.
  - ✓ **`readfile()`** – Rzeczą odróżniającą tą funkcję od instrukcji zawartych w poprzednim punkcie jest to, że dane pobrane z pliku nie są przetwarzane przez PHP, więc funkcja ta nadaje się tylko do wyświetlania plików HTML bądź czysto tekstowych.

Programowanie przykład 1



Programowanie przykład 1 - formularz



sk_1.php

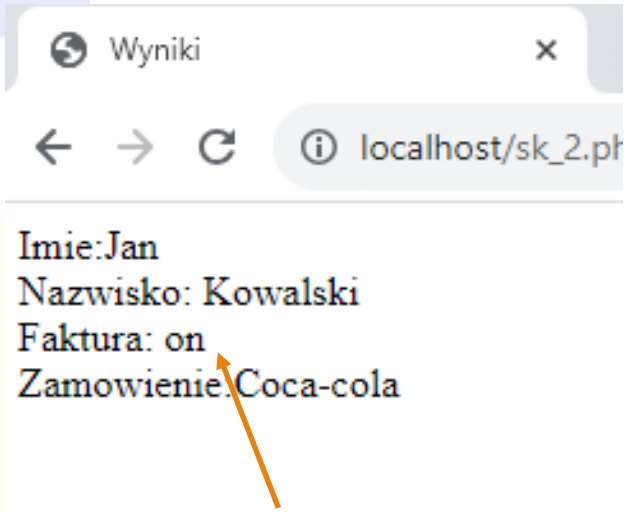
C:\Oprogramowanie\Apache24\htdocs\sk_1.php

```
<head>
  <title> Test formularza </title>
</head>
<body>
<form action="sk_2a.php" method="GET">
 Imie: <input type=text name="imie"/> <br/>
 Nazwisko: <input type=text name="nazwisko"/> <br/>
 Prosze o fakture do zamowienia:
 <input type=checkbox name="faktura"/> <br/>
 Chce zamowic:<br/>
  <input type=radio name="zamow" value="kola"/>Coca-coca<br>
  <input type=radio name="zamow"  value="fanta"/>Fanta<br>
  <input type=radio name=zamow"  value="sprite"/>Sprit<br>
  <input type=submit value="Wyslij"/>
 </form>
</body>
</html>
```

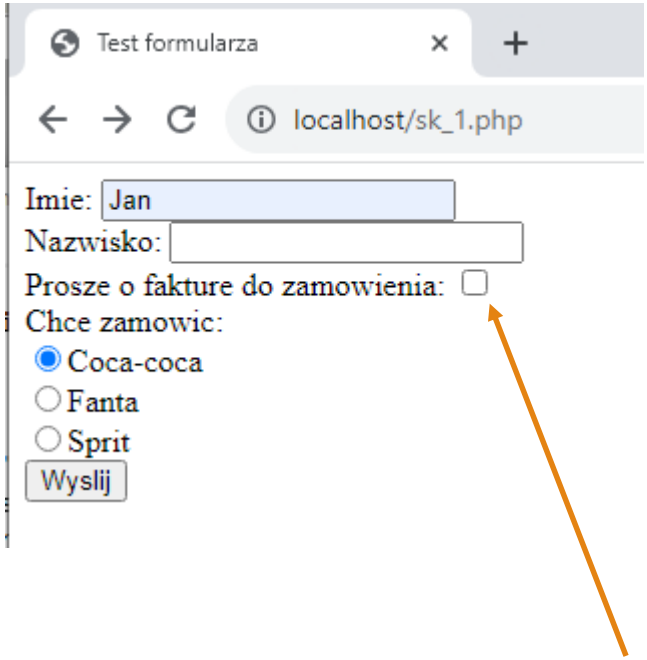
Programowanie przykład 1 – strona obsługi.

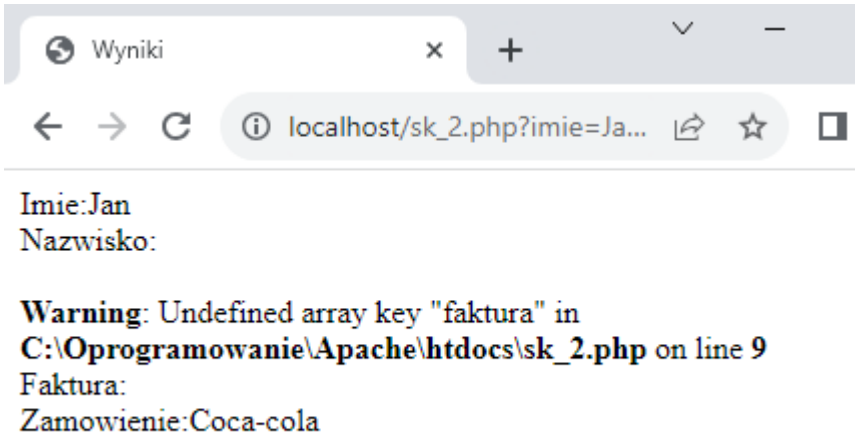
sk_2.php

```
1  <html>
2    <head>
3      <title> Wyniki </title>
4    </head>
5    <body>
6     <?php
7      echo "Imie:".$_GET['imie']."<br>";
8      echo "Nazwisko: $_GET[nazwisko] <br>";
9      echo "Faktura:  $_GET[faktura]  <br>";
10     echo " Zamowienie:";
11     switch($_GET['zamow'])
12     { case "kola"  :echo "Coca-cola"; break;
13       case "fanta" :echo "Fanta"; break;
14       case "sprite":echo "Sprite"; break;
15     };
16   ?>
17   </body>
18 </html>
19
```



Programowanie przykład 1 – strona obsługi.

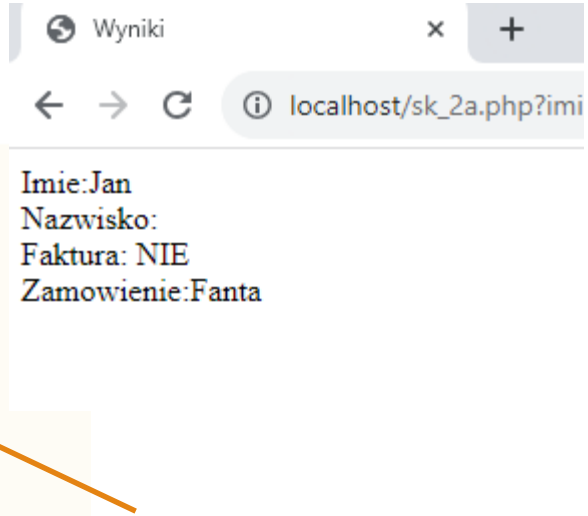




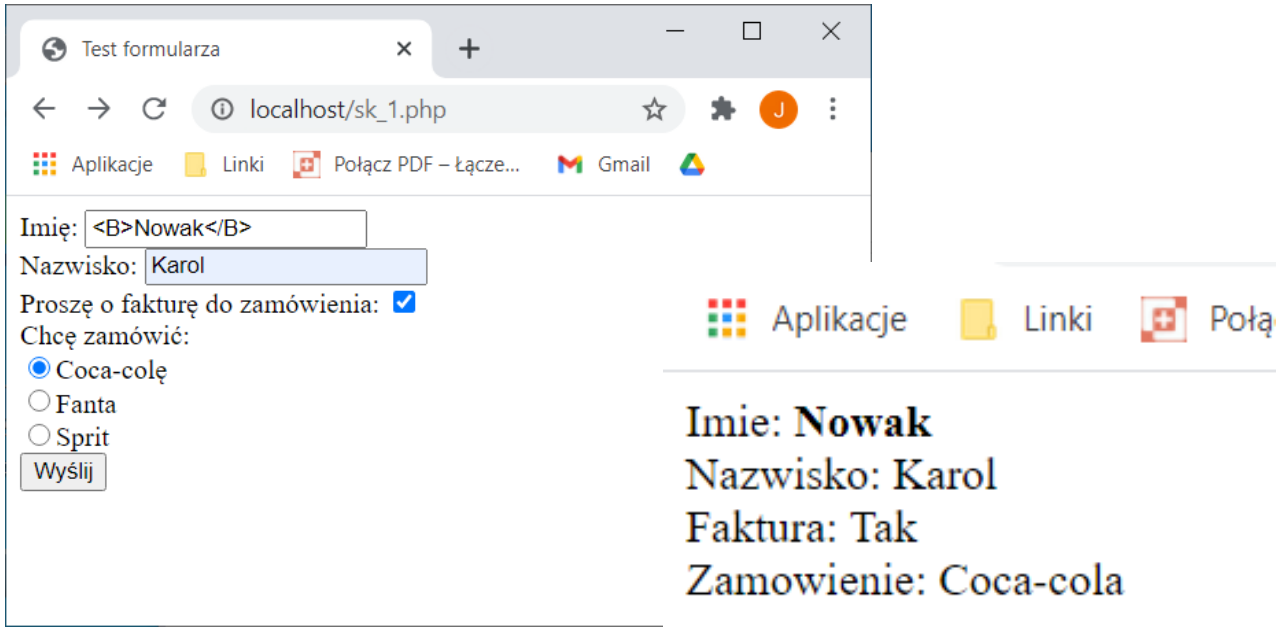
Programowanie przykład 1 – strona obsługi.

sk_2a.php

```
1  <html>
2    <head>
3      <title> Wyniki </title>
4    </head>
5    <body>
6     <?php
7      echo "Imie:".$_GET['imie']."<br>";
8      echo "Nazwisko: $_GET[nazwisko] <br>";
9      if (isset($_GET['faktura']))
10       echo "Faktura:  TAK  <br>";
11     else
12       echo "Faktura:  NIE  <br>";
13     echo " Zamowienie:";
14     switch($_GET['zamow'])
15     { case "kola"  :echo "Coca-cola"; break;
16       case "fanta" :echo "Fanta"; break;
17       case "sprite":echo "Sprite"; break;
18     };
19   ?>
20   </body>
21 </html>
22
```



Programowanie przykład 1 – strona obsługi - uwaga.



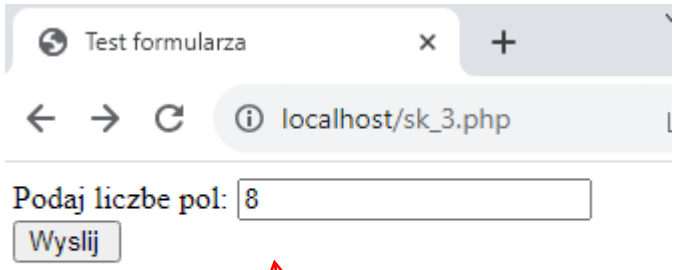
Było:

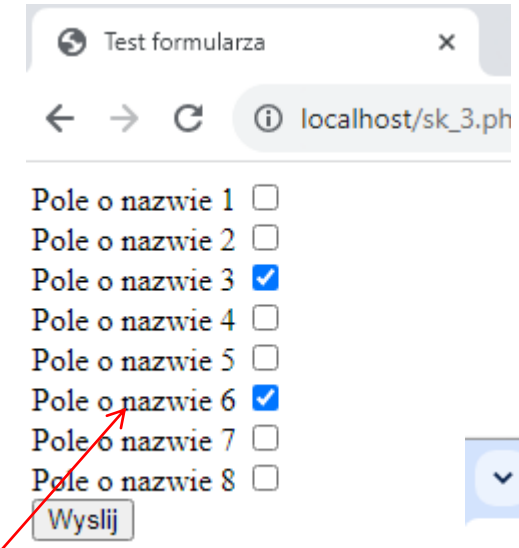
```
echo $_GET['nazwisko']
```

Zmieniamy na:

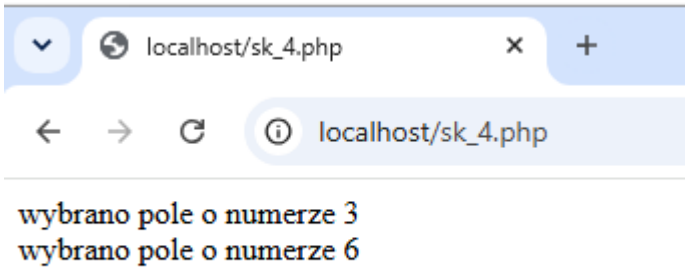
```
htmlspecialchars($_GET['nazwisko'])
```

Programowanie przykład 2





Strona 2



Strona 1

Programowanie przykład 2 – strona pierwsza.

sk_3.php

```
1  <html>
2   <head><title> Test formularza </title></head> </body>
3   <?php
4    if ($_SERVER["REQUEST_METHOD"]=="GET")
5    {
6      echo '<form action=" " method="post">
7      Podaj liczbe pol: <input type=text name="ile"/><br>
8     <input type=submit value="Wyslij "/>
9     </form>' ;
10   }
11   else
12   {
13   echo '<form action="sk_4.php" method="post">';
14    for($x=1;$x<=$_POST["ile"];$x++)
15   { echo "Pole o nazwie ".$x." " ;
16      echo '<input type=checkbox name="lista[]" value="'.$x.'"/></BR>';
17   };
18   echo '<input type=submit value="Wyslij"/></form>';
19   }
20  ?>
21  </body>
22 </html>
23
```

Programowanie przykład 2 – strona druga.

sk_4.php

```php
<?php
 for($x=0;$x<count($_POST["lista"]);$x++)
 { echo 'wybrano pole o numerze ' .$_POST["lista"][$x].'</br>';
 }
?>
```

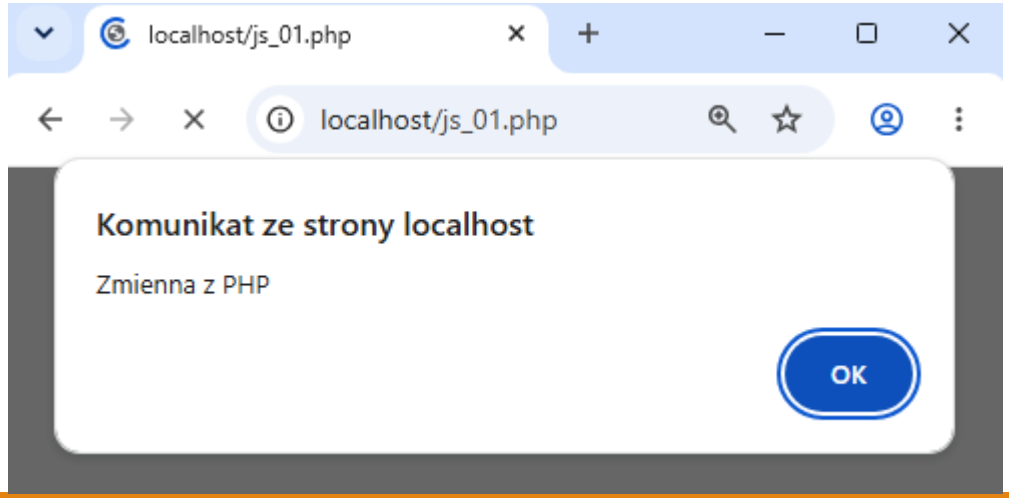
Programowanie

- PHP połączenie z JavaScript

js_01.php

```php
<?php
 $zmienna_php = "Zmienna z PHP";
?>

<script>
      let zmiennaJS = <?php echo json_encode($zmienna_php); ?>;
      alert(zmiennaJS);
</script>
```



Programowanie

- Przy wyjściu pytanie czy zapisać

js_02.php

```
1  <form id="f1" action="strona.php" method="POST">
2      <input type="text" name="imie" placeholder="Twoje imię">
3      <button type="submit" id="zap">Zapisz dane</button>
4  </form>
5
6  <script>
7      const p = document.getElementById('zap');
8      p.addEventListener( 'click', function()
9       {
10       if (confirm("Czy chcesz zapisać zmiany?")) { alert ("ZAPISZ")}
11       else { alert("Nie zapisuj");
12              event.preventDefault();
13        }
14      }
15     );
16
17 </script>
```

